Centro Universitário de Excelência
Sistemas de Informação

MecâniQA: persistência com JDBC e MySQL

Autores: Andrey Mota de Oliveira
Gabriel da Silva e Silva
Giovanna Oliveira Araújo
João Magalhães Mira Neto
Louise de Souza Santino

Feira de Santana, 2026

Agenda

O objetivo desta apresentação é demonstrar a evolução da API da MecâniQA para persistir dados no MySQL, utilizando JDBC e consultas SQL manuais.

1. Contextualização:
A evolução do MVP da MecâniQA na Fase 3.

2. Migração de dados:
Como o armazenamento temporário foi substituído pela persistência no banco.

3. Separação de camadas e fluxo dos dados
O papel do Controller, da Service e do Repository na aplicação.

4. Estrutura do banco e conexão JDBC:
Como as tabelas são criadas e como a aplicação acessa o MySQL.

5. Operações SQL e validações:
Conversão do ResultSet em objetos pelo Mapper e validação dos dados na camada Service.

6. Considerações Finais e Referências:
Principais resultados da implementação e materiais consultados.

Contextualização

- A **MecâniQA** é uma empresa de soluções voltada ao mercado automotivo.
- A equipe desenvolveu inicialmente o backend com um catálogo de peças e serviços e criou a Web API.
- Nesta etapa, implementamos o acesso ao banco de dados com JDBC e organizamos as operações nas camadas Service e Repository.



Migração de dados: memória

1 - Lista em memória

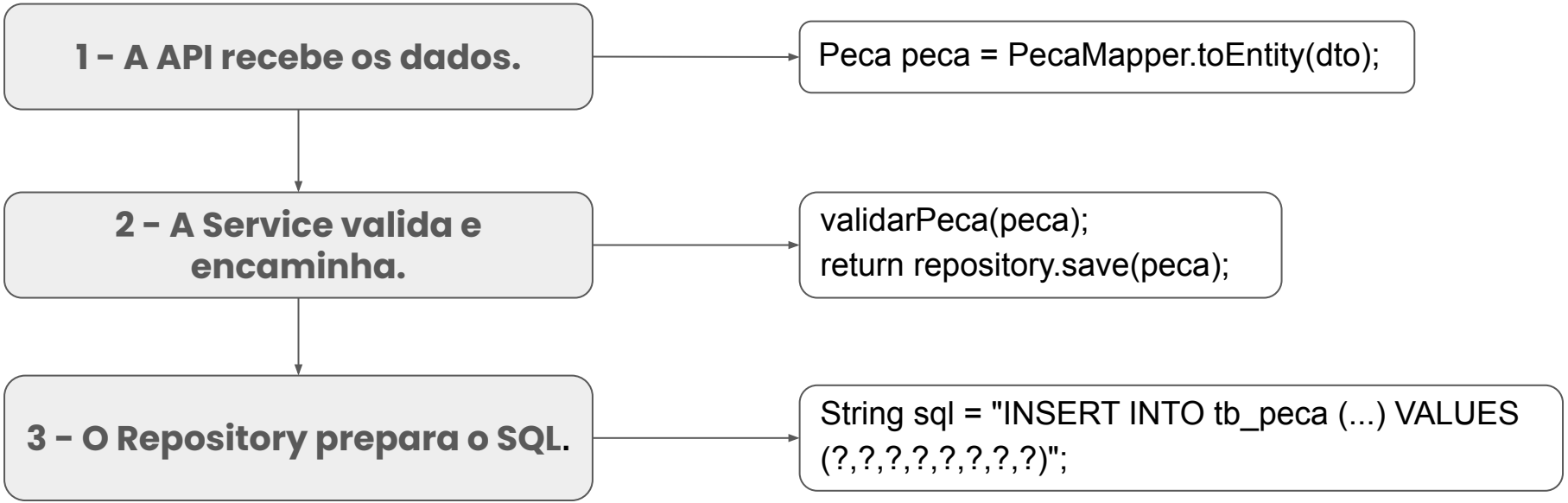
```
private final List<Peca> bancoEmMemoria = new ArrayList<>();
private int criarId = 1;
```

2 - Contador (criarId): gera um novo ID para cada peça que ainda não possui identificador.

```
public Peca postPeca(Peca peca)
{
        if (peca.getIdPeca() == null) {
            peca.setIdPeca(criarId++);
          }

        bancoEmMemoria.add(peca);
        return peca;
          }
```

Migração de dados: MySQL



Migração de dados: MySQL

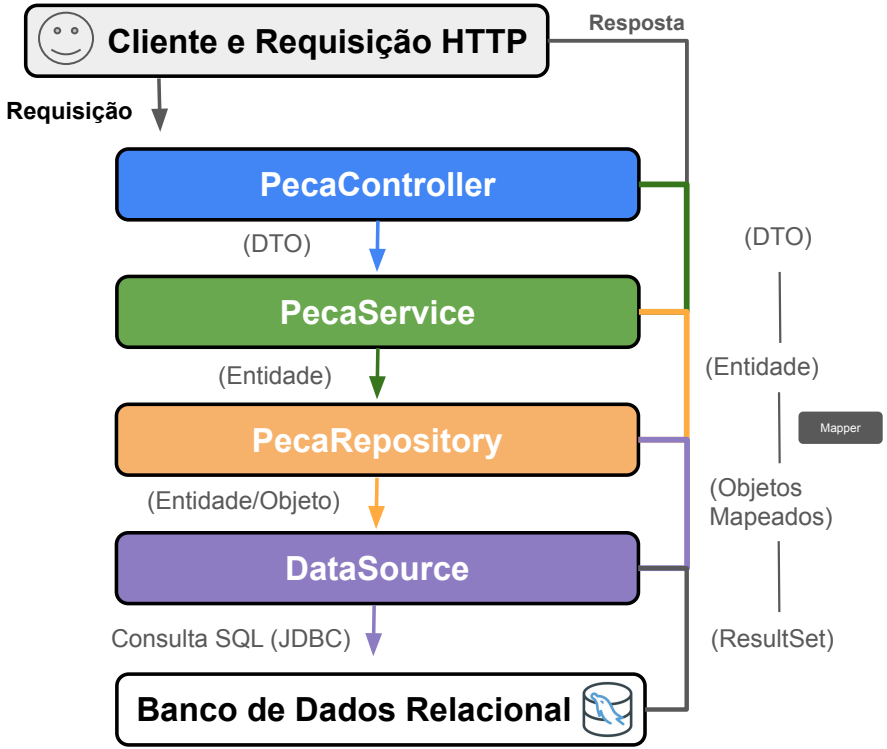
4 - JDBC abre a conexão e envia os parâmetros.

```
try (Connection conn = dataSource.getConnection();
     PreparedStatement ps =
         conn.prepareStatement(sql,
Statement.RETURN_GENERATED_KEYS)) {
    fill(ps, entity);
    ps.executeUpdate();}
```

5 - O MySQL grava os dados e gera o ID.

```
 try (ResultSet keys = ps.getGeneratedKeys()) {
    if (keys.next()) entity.setIdPeca(keys.getInt(1));
}
```

Arquitetura em Camadas



MecâniQA: persistência com JDBC e MySQL

Estrutura do banco de dados

```
CREATE TABLE IF NOT EXISTS tb_peca (
  peca_id INT NOT NULL AUTO_INCREMENT,
  cod_barras VARCHAR(13),
  quant_estoque INT,
  preco_custo DECIMAL(10,2),
  preco_venda DECIMAL(10,2),
  data_cadastro DATETIME NOT NULL DEFAULT
CURRENT_TIMESTAMP,
  PRIMARY KEY (peca_id)
);
```

`AUTO_INCREMENT` gera o identificador.

`PRIMARY KEY` identifica cada registro.

`DECIMAL(10,2)` guarda valores monetários.

`DEFAULT CURRENT_TIMESTAMP` preenche a data de cadastro.

Conexão JDBC → MySQL

- **Configuração da conexão**

```
spring.datasource.url=jdbc:mysql://localhost:3307/MECANIQA
spring.datasource.username=${DB_USER:root}
spring.datasource.password=${DB_PASSWORD}
```

Configuração	O que informa
`jdbc:mysql`	Protocolo JDBC e banco MySQL
`localhost:3307`	Servidor local e porta do MySQL
`MECANIQA`	Banco de dados usado pela aplicação
`DB_USER` / `DB_PASSWORD`	Credenciais configuradas no ambiente

Conexão JDBC → MySQL

`dataSource.getConnection()` — solicita uma conexão ao `DataSource`.

`conn.prepareStatement(sql)` — prepara a instrução SQL.

`ps.setInt(1, id)` — preenche o primeiro `?` com o ID recebido.

`ps.executeQuery()` — executa o `SELECT` e retorna um `ResultSet`

try-with-resources — fecha conexão, statement e resultado ao terminar.

```
String sql = "SELECT * FROM tb_peca
WHERE peca_id = ?";

try (Connection conn =
dataSource.getConnection();
    PreparedStatement ps =
conn.prepareStatement(sql)) {

ps.setInt(1, id);

try (ResultSet rs = ps.executeQuery()) {}
}
```

CRUD manual com JDBC

Operação	Exemplo no projeto	Execução JDBC
Criar	`INSERT INTO tb_peca`	`executeUpdate()`
Consultar	`SELECT ... WHERE peca_id = ?`	`executeQuery()`
Atualizar	`UPDATE tb_peca SET`	`executeUpdate()`
Excluir	`DELETE FROM tb_peca WHERE`	`executeUpdate()`

Validações

```
private void validarPreco(BigDecimal preco, String campo) {
  if (preco == null || preco.compareTo(BigDecimal.ZERO) < 0)
                                                    {
                throw new DadosInvalidosException(
          campo + " inválido: deve ser maior ou igual a zero."
                                                    );
                                                  } }
```

- **Não pode estar ausente.**
- **Não pode ser negativo.**
- **Se a regra não for atendida, a Service lança uma exceção personalizada.**

Considerações finais

- Transição da persistência temporária em memória para o banco relacional MySQL utilizando exclusivamente JDBC puro.
- Garantia do fluxo rígido em camadas (Controller → Service → Repository → JDBC), mantendo total desacoplamento e isolamento de regras de negócio.
- Implementação de PreparedStatement para prevenção contra *SQL Injection* e gestão segura de conexões com try-with-resources.

Referências

- **DEITEL, Paul; DEITEL, Harvey.** *Java: como programar*. 10. ed. São Paulo: Pearson Education do Brasil, 2016.

- **GAMMA, Erich et al.** *Padrões de Projeto: Soluções Reutilizáveis de Software Orientado a Objetos*. Porto Alegre: Bookman, 2000.

- **ORACLE.** *Java Database Connectivity (JDBC) API Documentation*. Redwood City: Oracle Corporation, 2026. Disponível em: https://docs.oracle.com/en/java/javase/docs/api/java.sql/module-summary.html.

- **MYSQL.** *MySQL 8.0 Reference Manual: Connector/J Developer Guide*. Oracle Corporation, 2026. Disponível em: https://dev.mysql.com/doc/connector-j/8.0/en/.